



Universidad Nacional Autónoma de México

Facultad de Ciencias

Práctica Número 3

Cristian Guillermo Mejia Cisneros

Número de cuenta: 427329650

24 Septiembre de 2026

1. Objetivos

- **Composición de documentos con L^AT_EX**: Usar Overleaf para crear un documento aplicando formato de texto, listas, secciones e inserción de imágenes.
- **Control de versiones**: Inicializar un repositorio con Git, registrar cambios mediante commits y sincronizar con un repositorio remoto en GitHub
- **Documentación**: Escribir un archivo README.md que documente el contenido de un repositorio y explique cómo usar el código que contiene.

2. Desarrollo

2.1. Parte 1: L^AT_EX

Para esta parte de la práctica, hemos creado un documento sencillo con Overleaf. Dentro de este archivo, hay un breve resumen acerca de la historia de **Doom Eternal**. Para desarrollar el documento, usamos los paquetes:

```
1 \documentclass[spanish,12pt,letterpaper]{article}
2 \usepackage[a4paper, margin=2cm]{geometry}
3 \usepackage[T1]{fontenc}
4 \usepackage{graphicx}
5 \usepackage{enumitem}
6 \usepackage{float}
7 \usepackage[spanish]{babel}
```

- `\documentclass`: Nos ayuda a darle formato al documento, diciendo que lo queremos en español, tamaño de letra 12, tamaño de página carta y que escribiremos un artículo.
- `\usepackage[a4paper, margin=2cm]{geometry}`: Mejora el formato del documento.
- `\usepackage[T1]{fontenc}`: Permite introducir texto sin que nos detecte errores por usar ciertas puntuaciones como la acentuación.
- `\usepackage{graphicx}`: Nos permite introducir figuras o imágenes.
- `\usepackage{enumitem}`: Ayuda con la correcta enumeración, tablas, etc, en el documento.
- `\usepackage{float}`: Nos ayuda a controlar de mejor manera, como ubicamos las imágenes que insertamos en el documento.
- `\usepackage[spanish]{babel}`: Le dice al documento que redactaremos en español.

Para la siguiente parte del texto, utilizamos:

```
1 \title{Doom Eternal}
2 \author{Cristian Guillermo Mejia Cisneros}
3 \date{\today}
```

- `\title{Doom Eternal}`: Nos brinda el título del documento.
- `\author{Cristian Guillermo Mejia Cisneros}`: Coloca el autor del documento.

- `\date{\today}`: Da la fecha actual de la realización del documento.

Con esto “Declaramos” la portada de nuestro escrito. No basta solo con escribir esto último para que aparezca en nuestro documento, la forma de insertarlo es, mandándolo a llamar (es decir, lo escribimos) como sigue:

```
1 \begin{document}
2     \maketitle
3     \newpage
4 \end{document}
```

Lo anterior indica que dentro de nuestro documento (todo entre las líneas `\begin{document}` y `\end{document}`) se inserta nuestra pequeña portada con los datos suministrados mediante el comando `\maketitle`; `\newpage` solo hace referencia a dar un salto de página, para que nuestro documento empiece a partir de la siguiente hoja y no junto con el título, lo anterior lo hemos realizado por meros fines estéticos.

Para no terminar alargando el reporte más de lo necesario, enlistaremos todos los comandos más “comunes”, que se utilizaron para escribir todo el documento $\text{\LaTeX}$:

- `\section{}`: Nos sirve para dividir el documento por secciones, enumerándolas de la forma (1.), (2.), ..., (n.), para tener mejor estructurado el escrito.
- `\subsection{}`: Crea una sección, dentro de las secciones creadas anteriormente, realizando una subdivisión del estilo (n.1), donde n es el número de sección en la que estemos posicionados mientras escribimos el documento de $\text{\LaTeX}$.
- `\noindent`: Por lo general, en $\text{\LaTeX}$, al escribir un documento, es usual que al empezar un nuevo párrafo, este nos de una pequeña tabulación en automático. Antes de redactar algo, escribimos este último comando para evitar esta pequeña tabulación.
- `\[1mm]`: Da un salto de línea de 1 milímetro.
- `\begin{itemize}`: Inicializa un enlistado que se ve de la siguiente manera:
 - objeto número 1
 - objeto número 2
 - objeto número 3

`\item` nos ayuda a separar cada cosa enlistada. Para terminar con la lista, escribimos `\end{itemize}`.

- `\begin{figure}`: En nuestro documento, para ingresar una imagen escribimos lo siguiente:

```
1 \begin{figure}[H]
2     \centering
3     \includegraphics[width=0.7\textwidth]{Slayer.jpg}
4     \caption{\textbf{DOOM Slayer}.}
5 \end{figure}
```

Donde:

- `\centering`: centra la imagen en la página del documento.

- `\includegraphics[width=0.7\textwidth]{Slayer.jpg}`: Inserta la imagen que queremos colocar en nuestro documento, llamada `Slayer.jpg` indicando el formato de nuestra imagen (en este caso `.jpg`) y que le daremos un tamaño de 0.7 en relación con su dimensión original. Para que el código funcione correctamente, la imagen debe estar dentro de nuestros archivos de Overleaf, o la carpeta donde se ubica el `.tex` del documento $\text{\LaTeX}$.
- `\caption{DOOM Slayer}`: Coloca un texto al pie de la imagen que la describe.
- `[H]`: En nuestro código, da la orden de colocar nuestra imagen, exactamente después de el texto que hemos escrito.

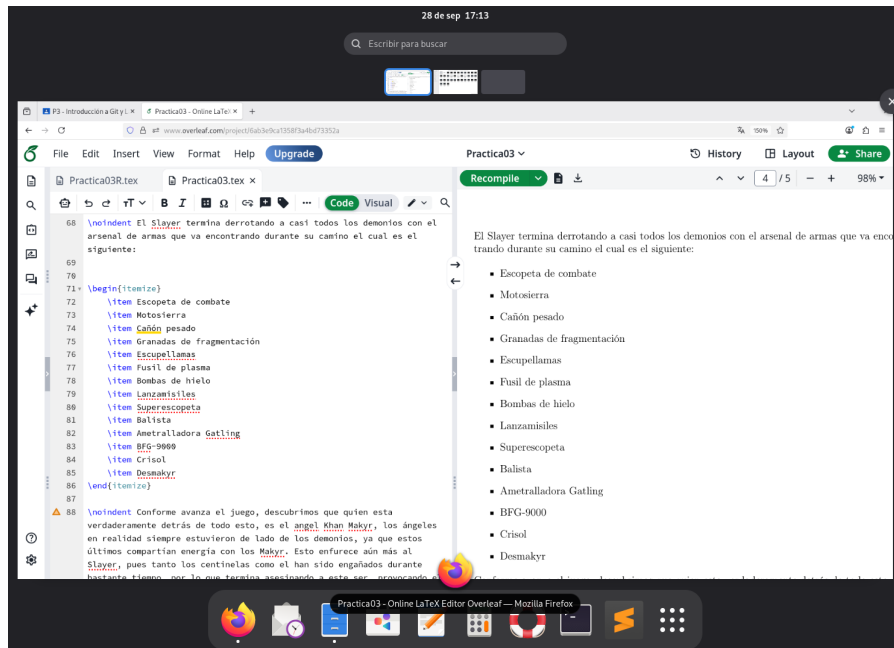


Figura 1: Código y PDF del documento creado en $\text{\LaTeX}$.

En la **Figura 1**, podemos ver una parte del código escrito para el documento $\text{\LaTeX}$.

2.1.1. Referencias

1. Bacilio Navarro, H. M. (6 de enero de 2017). *LaTeX usando texmaker Tutorial 0 - Instalación de Miktex y Texmaker Windows 8 y 10 - español* [Video]. YouTube. Video.

2.2. Parte 2: Git

1. Para configurar Git, antes que nada, primero tuvimos que instalarlo, ya que cuando empezamos con los comando de configuración, no sabia que realizar. Para instalarlo simplemente ejecutamos en la terminal de Linux:

```
1 sudo apt update
2 sudo apt install git
```

Posteriormente, configuramos nuestra cuenta y correo de git con los siguientes comandos en la terminal:

```
1 git config --global user.name "Guillermo Mejia"
2 git config --global user.email cgmc17@ciencias.unam.mx
```

Y con el siguiente comando en la terminal nos aseguramos que los datos sean correctos:

```
1 git config --list
```

2. Ya teníamos una cuenta Github creada mediante nuestro correo institucional, por lo que para este paso, no fue necesario realizar mucho, la **Figura 2** muestra la cuenta Github creada.

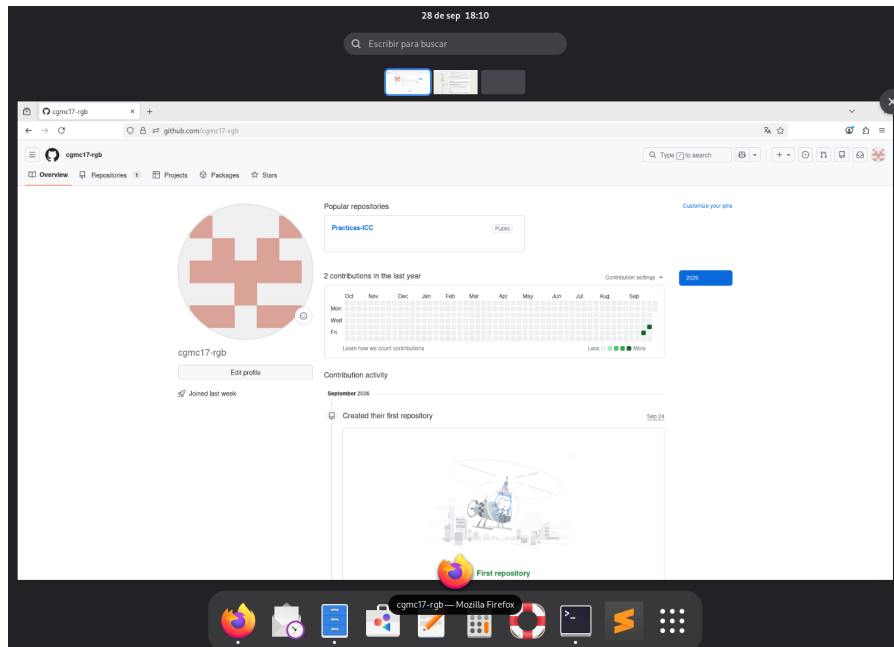


Figura 2: *Perfil de GitHub.*

3. En nuestro directorio:

/Documentos/FCiencias/Semestre_2027-1/ICC/Practicas/Repositorio

Producimos la carpeta **Repositorio** que es donde crearemos el repositorio Github. Ahora, dentro de /Documentos/FCiencias/Semestre_2027-1/ICC/Practicas/Repositorio, utilizamos el siguiente comando:

```
1 git init
```

Para indicar que esa carpeta será un repositorio de Git.

Es importante escribir el hecho de que antes de realizar el comando `git init` ejecutamos:

```
1 git config --global init.defaultBranch main
```

Indicando que rama principal será **main**. Una rama en git es como una linea del tiempo del que se puede desprender más ramas, en las cuales podemos modificar nuestro código, sin necesidad de alterar la rama principal **main**.

4. Dentro del directorio, vamos a crear un **README.md**, Que es un archivo el cual nos permite especificar de manera breve, el contenido de nuestro repositorio y detalla quien es el autor del mismo.

Dentro de este README.md, escribimos lo siguiente:

```
1 # Mi Primer README
2
3 ## Contenido
4
5 En el siguiente **Repositorio** podran ver el contenido de
6 las practicas hechas en Latex, asi como los avances que se
7 tengan en los distintos archivos de programacion creados.
8
9 Esperando no ser demasiado breve con el mensaje, doy gracias al
10 lector por echarle un ojo al documento.
11
12 ## Autor
13
14 **Cristian Guillermo Mejia Cisneros**
```

Cabe aclarar, si es necesario incluirlo, que README.md fue creado mediante `touch README.md` y modificado con `sudo nano README.md`. Creado este archivo y modificado, lo agregamos al repositorio mediante:

```
1 git add README.md
```

5. Con ayuda del comando:

```
1 git commit -m "Primer commit README"
```

realizamos nuestro primer commit, que es en pocas palabras y tomándonos la libertad de parafrasear, un momento del tiempo del repositorio, es decir, como una fotografía del estado en el que se encontraba, en ese preciso momento, un archivo. Prácticamente guarda la versión que tenía un archivo en un tiempo determinado, ¿Por qué nos interesa esto?, si en algún momento realizamos alguna modificación a nuestro archivo y por alguna razón ya no funciona, podemos regresar al estado donde todo funcionaba de pies a cabeza.

6. Ahora, vamos a enlazar nuestro repositorio Git, a GitHub, para ello nuestra cuenta GitHub debe estar registrada con el mismo correo que configuramos en Git, para nuestro caso `cgmcl7@ciencias.unam.mx` y dentro de nuestra cuenta creamos un repositorio con el nombre `Practicas-ICC` dejándolo publico para que pueda corroborarse que se realizo.

El primer paso es crear un **token** de seguridad para nuestro repositorio de GitHub. Dentro de la página de GitHub, vamos acceder, desde la parte de nuestro perfil, a **settings**, seguido vamos a buscar la opción llamada **Developer settings** dando igual click para acceder a esta parte. Ahí, encontraremos una pestaña llamada **Personal access tokens** donde al darle click nos desplegará algunas opciones de las cuales elegiremos la que dice **Tokens (classic)**.

Realizado los pasos anteriores, damos click en la parte de **Generate new token** seguido de **Generate new token (classic)**, lo cual nos llevará a una nueva página donde podremos colocar el nombre del token, la fecha de expiración del mismo también, elegimos las opciones **repo** que nos permitirán tener control total de los repositorios tanto públicos como privados y finalmente, una ves seleccionadas y debidamente llenadas las opciones,

daremos click en **Generate token**, esto nos terminará dando un código de “acceso” a nuestros repositorios, cuando intentemos acceder a ellos mediante Git en nuestra terminal, es muy importante guardar este código y no perderlo pues es de ocasión única para copiarlo. En la **Figura 3** podremos ver una captura de una parte del proceso realizado.

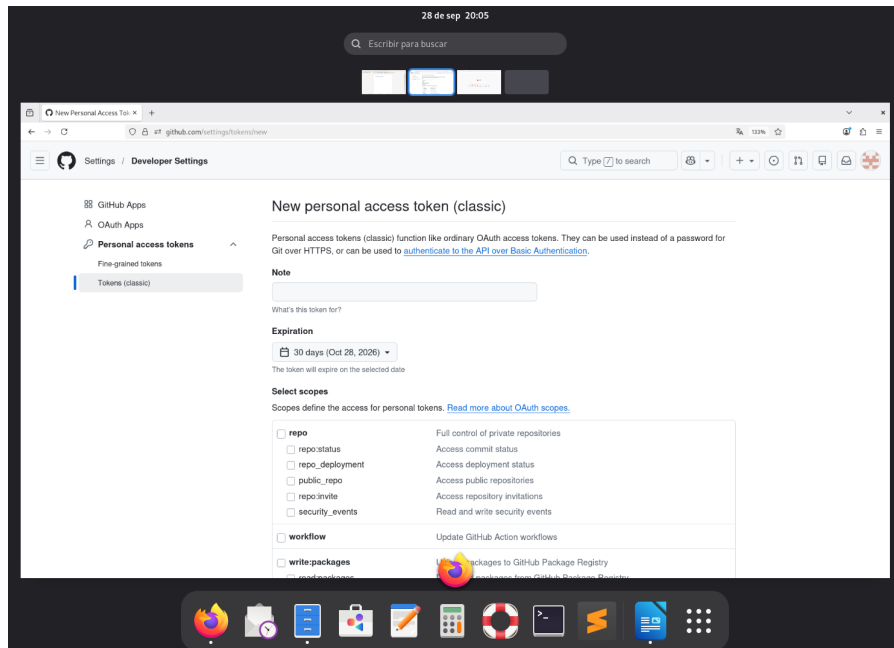


Figura 3: *Creación de token para enlazar repositorio.*

Posteriormente a esto, ahora, si accedemos a nuestro repositorio creado en GitHub, podremos ver un link para acceder a el, mismo que copiaremos y utilizaremos en nuestra terminal de la siguiente forma:

```
1 git remote add origin https://github.com/cgmc17-rgb/Practicas-ICC
```

Comando que conecta los repositorios, tanto el creado en GitHub como en Git. Posterior a eso ejecutamos en la terminal:

```
1 git push -u origin main
```

que termina por sincronizar todo de manera definitiva. Al ejecutar el comando, nos pedirá nuestro nombre de usuario de GitHub y después el token que hemos creado correctamente, si todo sale bien obtendremos algo parecido a la **Figura 4**.

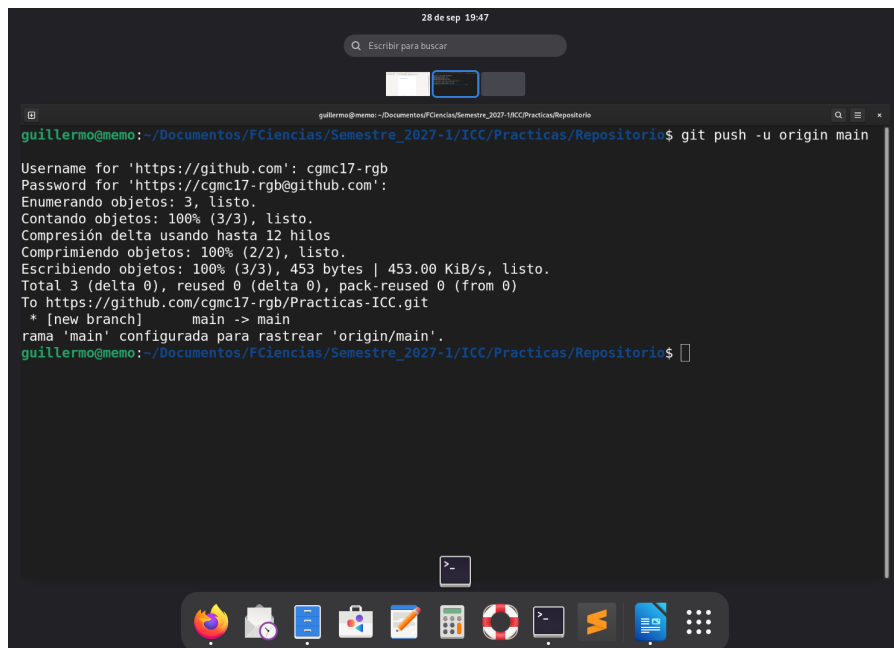


Figura 4: Conexión exitosa de los repositorios de GitHub y Git.

Finalmente en la **Figura 5** podremos observar que en efecto, el archivo README.md ya aparece en nuestro repositorio, confirmando la sincronización.

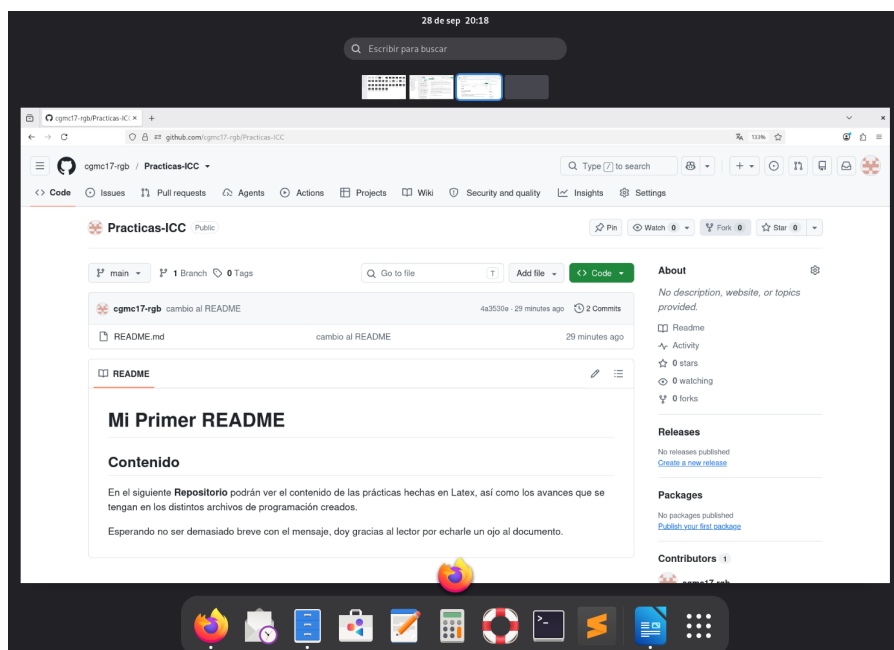


Figura 5: Repositorio GitHub ya con los archivos del Repositorio de Git.

Guardando el documento $\text{\LaTeX}$

1. Dentro del directorio en donde está ubicado nuestro repositorio, agregamos una carpeta nueva llama Practica03 y dentro de esta misma un subdirectorio llamado Latex, en el, vamos a guardar los archivos generados por nuestro documento Latex.
2. Copiamos los archivos $\text{\LaTeX}$, el PDF y las imágenes que usamos dentro del documento de $\text{\LaTeX}$, y en la carpeta del mismo nombre, pegamos los archivos. Para obtener todos

estos archivos, en Overleaf damos click en File, luego vamos a donde dice descargar, para posteriormente bajar el .zip, lo guardamos en la carpeta descargas y lo descomprimos obteniendo todos los archivos necesarios para que L^AT_EX ejecute correctamente.

3. Ahora, cargamos bien estos archivos en el repositorio con:

```
1 git add Practica03.tex
2 git add Slayer.jpg
3 git add Practica02.pdf
```

Después ejecutamos:

```
1 git commit -m "Latex e imagenes agregados"
```

Para que finalmente escribamos en la terminal:

```
1 git push
```

Que sincronizará de nuevo toda la información del repositorio de Git con el de GitHub.

4. Solo resta verificar que todo está en orden tanto en Git como en GitHub. En la **Figura 6**, podemos ver una captura del repositorio actualizado con los archivos anteriormente agregados.

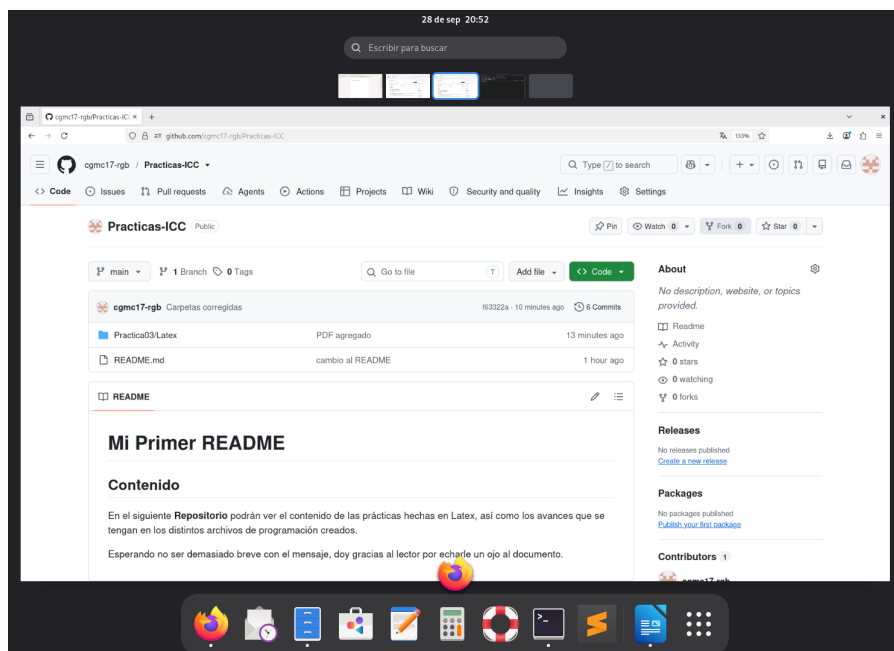


Figura 6: Nuevos archivos agregados al repositorio y visibles en GitHub.

Para verificar que todo este correcto en Git ejecutamos:

```
1 git log --oneline
```

Que nos mostrará una imagen de los commits creados. Notaremos que son alrededor de 5 commits, esto se debe al hecho de que mientras se realizaba la práctica, se tuvieron algunos errores tanto al escribir el README.md como al acomodar los archivos en las carpetas correspondientes. En la **Figura 7** podemos ver los commits creados.

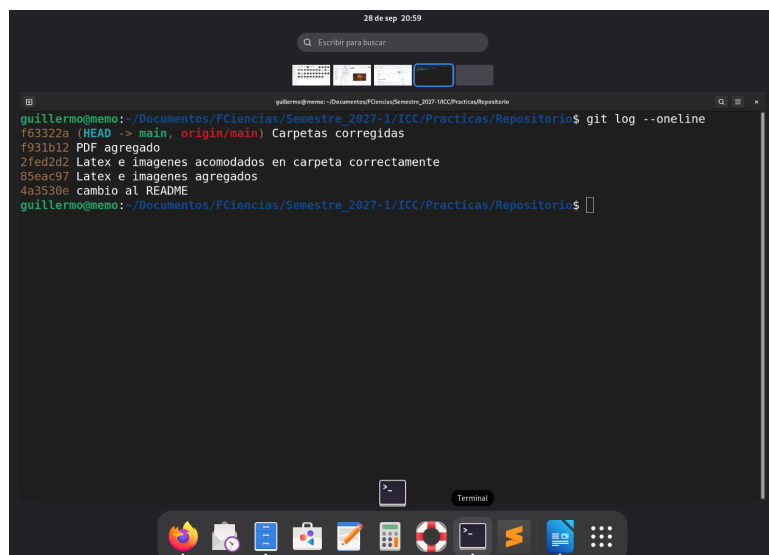
A screenshot of a terminal window on a Linux system. The terminal shows the output of the command 'git log --oneline' in a repository. The output lists five commits with their hashes and descriptions. The commit hashes are f63322a, f931b12, 2fed2d2, 85eac97, and 4a3530e. The descriptions are 'Carpetas corregidas', 'PDF agregado', 'Latex e imagenes acomodados en carpeta correctamente', 'Latex e imagenes agregados', and 'cambio al README' respectively. The terminal prompt is 'guillermo@memo:~/Documentos/FCiencias/Semestre_2027-1/ICC/Practicas/Repositorio\$'. The terminal window is titled 'Terminal' and has a search bar at the top with the text 'Escribir para buscar'.

Figura 7: *commits* realizados.

2.2.1. Referencias

1. DreamHost. (16 de julio de 2026). Configurar git para usar 'main' como rama principal. Centro de Ayuda de DreamHost. Página Web.
2. Guo, D. (s. f.). Make a README. Recuperado el 28 de septiembre de 2026, de Página Web.
3. Chacon, S. y Straub, B. (2014). Capítulo 1.6: Configurando Git por primera vez. En Pro Git (2.^a ed.). Apress. página web.

3. Conclusión

El documento anterior, recopila toda la información relacionada a lo realizado en la **Práctica Número 3**. Utilizamos Overleaf, para generar documentos en $\text{\LaTeX}$, que si bien en este momento en específico de la carrera, parece una “pérdida de tiempo” aprender un procesador de texto distinto a los convencionales (word, canva, etc.), lo cierto es, que a futuro, la herramienta $\text{\LaTeX}$ brinda una mayor consistencia y estructura en los escritos; dota de un estilo profesional a los documentos, reflejando profesionalismo en la creación de trabajos, desde los más comunes hasta los terminales (como lo puede ser una tesis).

Así mismo, la herramienta Git en conjunto con GitHub, nos permite tener un mayor control sobre nuestro código, ya sea tanto de $\text{\LaTeX}$ (como en nuestros ejemplos de la práctica) o de cualquier otro lenguaje de programación, obteniendo la seguridad de que si algo falla, podemos consultar los respaldos guardados en nuestros repositorio.

Finalmente, el desarrollar un texto tipo `README.md`, termina por culminar, una organización impecable de nuestros archivos, pues sumado a $\text{\LaTeX}$, Git y GitHub, `README` ayuda a tener un contexto de lo que hemos hecho a lo largo de nuestro repositorio, tanto para entenderlo si lo consultamos después de mucho tiempo, así como para quien accede a nuestro espacio y trabaja en equipo con nosotros, entendiendo en que consiste la información dentro del repositorio, así como los cambios que se hagan a futuro.